

TankOS 1

Firmware de contrôle pour plateforme chenillée radiocommandée

Projet Tank Terminale, Spécialité Sciences de l'Ingénieur, session 2026/2027 — Version 1 — Carte cible Arduino Mega 2560 (ATmega2560)

Sommaire

1. Présentation générale
2. Architecture matérielle
3. Architecture logicielle et ordonnancement FreeRTOS
4. Module Traction
5. Module Tourelle
6. Sécurité et failsafe
7. Supervision batterie et courant
8. Contrôle d'accès admin (RFID)
9. Module Communication (Série / Bluetooth)
10. Module Système
11. Référence des commandes Série et Bluetooth
12. Compilation et déploiement
13. Annexe : brochage complet

1. Présentation générale

TankOS est le firmware embarqué pilotant une plateforme chenillée radiocommandée à deux moteurs de traction et une tourelle motorisée équipée d'un système d'armement simulé (ESC double). Le firmware est écrit en C++ pour l'environnement Arduino, compilé et déployé via PlatformIO, et s'exécute sur une carte Arduino Mega 2560 basé sur le microcontrôleur ATmega2560 (8 bits, 16 MHz, 8 Ko de SRAM, 256 Ko de mémoire programme).

Le système repose sur un noyau temps réel FreeRTOS (portage AVR de la bibliothèque feilipu/FreeRTOS) et organise le traitement en cinq tâches concurrentes ordonnancées par priorité. Cette architecture remplace une structure séquentielle classique de type `loop()` et garantit que les fonctions critiques de sécurité (perte de liaison radio, surintensité, décharge batterie) sont exécutées avec une latence bornée, indépendamment de la charge des autres sous-systèmes.

Les fonctions principales assurées par le firmware sont les suivantes :

- Pilotage différentiel de la traction par relais et potentiomètres numériques SPI
- Pilotage de la tourelle : élévation et rotation par servomoteurs, activation de deux contrôleurs de vitesse électroniques (ESC)
- Réception de dix voies de la radiocommande par capture d'impulsions PWM en interruption
- Supervision de deux batteries (châssis et ee) et d'un capteur de courant, avec bridage automatique de la vitesse et alertes
- Coupure matérielle de l'alimentation moteur en cas de défaillance (MOSFET de puissance piloté logiquement)
- Contrôle d'accès par badge RFID pour déverrouiller les commandes sensibles
- Interface de commande textuelle disponible simultanément sur liaison série USB et sur module Bluetooth HM-10

- Régulation thermique par ventilateur piloté en PWM matériel haute fréquence

2. Architecture matérielle

L'ensemble du brochage est centralisé dans le fichier `config.h`, seul point de référence pour toute modification physique du câblage. Les sous-systèmes matériels sont décrits ci-dessous ; le détail exhaustif des broches figure en annexe.

2.1 Chaîne de traction

Chaque moteur de traction est commandé par un ensemble de relais montés sur une carte pilote MDRVCT60A002M. La direction (avant/arrière) de chaque moteur est sélectionnée par une paire de relais dédiés, et deux relais supplémentaires assurent la coupure générale de l'alimentation de chaque côté (fonction de sécurité indépendante de la direction). La vitesse est réglée par un potentiomètre numérique SPI MCP41100 par moteur, adressé par une broche `CS` dédiée (42 pour le moteur gauche, 43 pour le moteur droit), le bus SPI (SCK/MOSI/MISO sur les broches matérielles 52/51/50) étant partagé avec le lecteur RFID.

2.2 Tourelle

La tourelle intègre trois servomoteurs analogiques pour le mouvement d'élévation et la rotation (broches 2, 3 et 6 pilotés en parallèle) et deux ESC pour l'armement (broches 4 et 5, pilotage en largeur d'impulsion 1000 à 2000 microsecondes)

2.3 Réception radiocommande

Une radiocommande de type huit voies transmet des signaux PWM captés par interruption logicielle sur changement d'état (bibliothèque `enableInterrupt`). Les voies exploitées couvrent la direction et la vitesse de traction, l'orientation et l'élévation de la tourelle, l'armement, la vitesse de rampe (VRB) et deux interrupteurs auxiliaires (SWC, SWD). La capture est réalisée dans `ISR.cpp`, fichier isolé du reste du firmware car il gère les temporisations critiques en contexte d'interruption.

2.4 Lecteur RFID

Le contrôle d'accès admin repose sur un lecteur RC522 en liaison SPI, avec sélection sur la broche 41 et réinitialisation sur la broche 40. Sur Mega, le bus SPI matériel est situé sur les broches 50/51/52 (contrairement à un Uno où il se trouve sur 11/12/13), et des résistances série de 330 ohms sont insérées sur SCK et MOSI pour la compatibilité de niveaux logiques 5V du module.

2.5 Liaison Bluetooth

Un module HM-10 est connecté sur `Serial1` à 9600 bauds et fournit une interface de commande sans fil identique à la liaison série USB. Une broche BRK (37) permet de forcer une rupture de connexion Bluetooth de façon logicielle.

2.6 Capteurs et actionneurs auxiliaires

La température est relevée par un capteur I2C MCP9808. Deux ponts diviseurs analogiques mesurent la tension des batteries châssis (A1) et tourelle (A2), un capteur à effet Hall (type ACS) mesure le courant consommé (A0). Un ventilateur est commandé en modulation de largeur d'impulsion matérielle à 25 kHz par configuration directe du timer 2 (broche 9). Une LED RGB de statut (broches 46 à 49) restitue l'état global du système.

2.7 Sécurité matérielle

Un transistor MOSFET canal P (FQP27P06) commandé sur la broche 32 permet la coupure matérielle de l'alimentation de puissance de la chaîne de traction, indépendamment de l'état des relais de direction, en complément de la logique de sécurité logicielle.

3. Architecture logicielle et ordonnancement FreeRTOS

Le firmware utilise le portage AVR de FreeRTOS (feilipu/FreeRTOS) afin de garantir un ordonnancement préemptif par priorité. Cinq tâches sont créées au démarrage, chacune encapsulant une boucle infinie rythmée par `vTaskDelay()` , ce qui libère le processeur pour les tâches de priorité inférieure ou égale pendant l'attente.

Tâche	Priorité	Période	Responsabilité
TaskSafety	4 (la plus haute)	20 ms	Détection de perte de liaison radio, calcul du facteur de bridage batterie, coupure de l'alimentation traction, gestion de la LED de statut
TaskTraction	3	34 ms	Lecture des voies radio et application des consignes de vitesse/direction à la chaîne de traction
TaskTourelle	3	34 ms	Élévation, rotation et armement de la tourelle
TaskCommunication	2	20 ms	Traitement des commandes série/Bluetooth, gestion des ventilateurs, télémétrie périodique
TaskAdmin	1 (la plus basse)	50 ms	Scrutation du lecteur RFID et gestion du verrouillage admin

La priorité de `TaskSafety` est volontairement maximale : elle doit pouvoir préempter toute autre tâche afin de garantir la coupure immédiate de la traction et de la tourelle en cas de perte de signal, quelle que soit la charge des autres modules.

3.1 Synchronisation par mutex

Deux sémaphores mutex, déclarés globalement dans `config.h` et instanciés dans `main.cpp` , protègent les ressources partagées entre tâches :

- `spiMutex` : protège l'accès exclusif au bus SPI matériel, partagé entre le lecteur RC522 (`admin.cpp`) et les potentiomètres numériques MCP41100 (`traction.cpp`). La totalité d'une transaction (sélection du composant, transfert des octets, désélection) est encadrée par la prise et la libération du mutex, afin de garantir son atomicité.
- `stateMutex` : protège l'état partagé calculé par `TaskSafety` (indicateur de failsafe actif, facteur de bridage de vitesse) et consommé par `TaskTraction` et `TaskTourelle` .

Les prises de mutex sont systématiquement bornées dans le temps (`pdMS_TO_TICKS(5)`) : en cas d'indisponibilité, la tâche appelante renonce à l'opération pour le cycle courant plutôt que de bloquer indéfiniment, préservant ainsi le respect des échéances temporelles de l'ordonnanceur.

3.2 Communication inter-tâches

Les données de la radiocommande sont exposées par `getRecepteurData()` , qui recopie sous section critique (`taskENTER_CRITICAL()` / `taskEXIT_CRITICAL()`) les variables volatiles renseignées par les routines d'interruption vers une structure `RcData` locale à chaque tâche appelante. Cette structure regroupe les huit canaux : axes de traction, axes de tourelle, interrupteurs SWC/SWD, potentiomètres VRA/VRB.

3.3 Point d'entrée et supervision

La fonction `setup()` initialise séquentiellement chaque module matériel, crée les mutex, instancie les cinq tâches via `xTaskCreate()` en vérifiant systématiquement leur code de retour, puis démarre l'ordonnanceur via `vTaskStartScheduler()`. La fonction `loop()` Arduino reste vide, le contrôle étant entièrement cédé au noyau FreeRTOS. Un hook `vApplicationStackOverflowHook()` est enregistré pour signaler sur le port série tout dépassement de pile d'une tâche et figer volontairement le système, ce comportement étant préférable à une corruption mémoire silencieuse.

4. Module Traction

Le module `traction.cpp` implémente une machine à états par moteur (structure `Motor`), avec trois états : `IDLE` (moteur arrêté), `RUNNING` (moteur en mouvement dans une direction donnée) et `SWITCHING` (phase transitoire lors d'une inversion de direction). Un changement direct de sens de marche sans passage par un arrêt intermédiaire de 200 millisecondes est proscrit afin de protéger l'électronique de puissance et les moteurs des courants d'appel liés à une inversion brutale.

```
struct Motor {
    int pinFwd, pinBck, pinCS;
    DirState currentDir;
    MotorState state;
    int currentPot;
    TickType_t switchStartTick;
};
```

La consigne de vitesse (0 à 100 pourcent) est convertie en valeur de résistance numérique par la fonction `setPot()`, qui transmet la commande d'écriture 0x11 suivie de la valeur mappée sur 0 à 255 au MCP41100 correspondant, sous protection du mutex SPI.

4.1 Lissage des commandes (rampe)

Les consignes brutes de vitesse et de direction ne sont pas appliquées instantanément : elles sont lissées dans le temps par un algorithme de rampe qui limite la variation maximale par cycle (`maxDelta`), calculée à partir du delta temps réel écoulé entre deux appels et d'un facteur de rampe réglable en direct par le potentiomètre VRB de la radiocommande (mappé de 1 à 100 pourcent). Ce mécanisme permet à l'opérateur d'ajuster la réactivité de la traction, d'une accélération quasi instantanée à une accélération progressive, sans modification du firmware.

4.2 Mixage différentiel

La direction et la vitesse sont combinées par un mixage différentiel classique : l'inclinaison en virage réduit la vitesse du moteur intérieur au virage tout en conservant la vitesse du moteur extérieur, permettant un pivotement sur place lorsque la commande de direction est maximale et la vitesse nulle.

4.3 Mode diagnostic

Un mode diagnostic accessible par commande admin permet de piloter un moteur de façon isolée (côté et vitesse imposés), en s'affranchissant du mixage et du lissage, utile pour des essais au banc sans récepteur radio connecté. Ce mode est explicitement exempté de la coupure matérielle par MOSFET de sécurité liée à la présence du signal radio, afin de rester utilisable hors ligne.

5. Module Tourelle

L'élévation est asservie en position absolue : l'angle des deux servomoteurs (synchronisés) est calculé par une mise à l'échelle linéaire de la plage brute de la voie radio correspondante vers l'intervalle 0 à 180 degrés.

La rotation azimutale est pilotée par un servomoteur.

L'armement (deux ESC) suit une logique d'activation par seuil : en dessous du seuil bas de la voie radio dédiée, les deux ESC reçoivent une commande d'arrêt (1000 microsecondes) ; au dessus, la commande brute est retransmise directement (contrainte entre 1000 et 2000 microsecondes), procurant un contrôle proportionnel de la puissance délivrée.

6. Sécurité et failsafe

6.1 Détection de perte de liaison radio

La fonction `checkFailsafe()` compare l'horodatage du dernier front descendant reçu sur la voie Y (méorisé en contexte d'interruption, `lastRcSignalTick`) au compteur de ticks courant. Un dépassement de 200 millisecondes sans nouvelle impulsion déclenche l'état de failsafe. Cette vérification est effectuée par `TaskSafety` à chaque cycle de 20 millisecondes.

6.2 Bridage dynamique par l'état batterie

Indépendamment du failsafe radio, `calculerBridageBatterie()` calcule un facteur multiplicatif de vitesse compris entre 0,3 et 1,0 en fonction du pourcentage de charge de la batterie châssis : la vitesse maximale autorisée décroît linéairement lorsque le pourcentage descend sous 20 pourcent, jusqu'à un plancher de 30 pourcent de la vitesse commandée. Ce facteur est combiné à la limite de vitesse réglable par commande (`mspd`) et appliqué multiplicativement dans `TaskTraction` aux consignes de vitesse et de direction.

6.3 Comportement en situation de failsafe

Lorsque le failsafe est actif, `TaskSafety` exécute dans le même cycle l'arrêt de la traction, l'arrêt de la tourelle, la coupure de l'alimentation de puissance des moteurs de traction via les relais dédiés, et l'affichage d'une couleur d'alerte (jaune) sur la LED de statut. Ce comportement est prioritaire sur toute commande utilisateur reçue par ailleurs, les tâches de traction et de tourelle interrogeant systématiquement l'état de failsafe partagé avant d'appliquer une quelconque consigne.

6.4 Alertes de courant et de batterie

Un facteur de sécurité indépendant est également calculé pour la surintensité (seuil de 18 ampères, correspondant approximativement à 1,4 fois le courant nominal en charge mesuré à 12,5 ampères). Les facteurs batterie et courant sont combinés par une fonction `min()` pour obtenir le facteur de sécurité global appliqué à la traction ; un dépassement de courant entraîne un bridage sévère (facteur 0,2) sans coupure totale. Les codes d'alerte diffusés simultanément sur les deux canaux série (`W0BATT`, `E0CURR`, `I0BATT`, `I0CURR`) sont soumis à une logique anti-inondation, avec répétition limitée à un message toutes les trois secondes et émission d'un message unique de retour à la normale.

7. Supervision batterie et courant

Le module `batterie.cpp` convertit les lectures analogiques en tension réelle par application d'un facteur d'échelle correspondant au rapport du pont diviseur (multiplicateur 6), puis en pourcentage de charge par interpolation linéaire sur une plage de tension propre à chaque batterie (châssis : 19,8 à 25,2 volts ; tourelle : 9,9 à 12,6 volts), le résultat étant systématiquement contraint entre 0 et 100 pourcent.

Le capteur de courant fonctionne par mesure différentielle autour d'une tension de repos (zéro ampère), calibrable à chaud par la commande `tare ampSensor`. La calibration moyenne 20 échantillons consécutifs avec une temporisation non bloquante entre chaque lecture (`vTaskDelay` lorsque l'ordonnanceur est actif, repli sur `delay()` sinon, pour rester utilisable avant le démarrage du scheduler).

8. Contrôle d'accès admin (RFID)

Le module `admin.cpp` gère l'authentification par badge RFID RC522 et le verrouillage des commandes sensibles. Une table statique d'UID autorisés est comparée octet à octet à chaque lecture de badge valide. La présentation d'un badge reconnu bascule l'état de déverrouillage admin (verrouillé/déverrouillé), avec une temporisation de deux secondes entre deux bascules successives pour éviter un rebond logiciel lié à une lecture répétée du même badge. La scrutation du lecteur est volontairement limitée à une fois toutes les 250 millisecondes afin de ne pas monopoliser le bus SPI partagé avec la traction.

La liste des commandes nécessitant un déverrouillage admin est centralisée dans `isAdminCommand()` et comprend les commandes `motor`, `reboot`, `tare`, `admin`, ainsi que `fanrange` à l'exception explicite de son argument `show` (lecture seule, non sensible). Toute tentative d'exécution de ces commandes sans déverrouillage préalable est rejetée par le parseur de commandes avec le code d'erreur `E0AUTH`.

9. Module Communication (Série / Bluetooth)

Le module `communication.cpp` centralise l'intégralité de l'interface de commande textuelle du tank sur un point d'entrée unique, `processCommand()`, appelé indifféremment pour les trames reçues sur le port série USB (`Serial`) et sur le module Bluetooth HM-10 (`Serial1`). Ce choix architectural élimine toute duplication de logique entre les deux canaux : l'ajout ou la modification d'une commande n'est réalisé qu'à un seul endroit.

La sortie de chaque commande est dirigée vers le canal d'origine de la requête grâce à l'abstraction `Print&` du framework Arduino, `Serial` et `Serial1` étant tous deux compatibles avec cette interface sans code spécifique à chacun.

9.1 Machine à états de rupture Bluetooth

La commande `btbrk` déclenche une séquence non bloquante de rupture de connexion Bluetooth, implémentée par une machine à états à trois phases (`BT_NORMAL`, `BT_WAIT_PRINT`, `BT_BREAK_LOW`) pilotée par comparaison de ticks, sans appel à `delay()`, ce qui garantit que la tâche de communication continue de traiter les autres canaux pendant la temporisation.

9.2 Régulation thermique

Le pilotage du ventilateur dépend de la position de la voie radio SWD et fonctionne selon trois régimes : arrêt complet en dessous de 1250, mode manuel entre 1250 et 1750 (vitesse fixée par la commande `fanspd`), mode automatique au dessus de 1750 (vitesse calculée par interpolation linéaire de la température mesurée entre les bornes réglables `dynTempMin` et `dynTempMax`, ajustables via la commande `fanrange`). La commande PWM est appliquée directement au registre `OCR2B` du timer 2, configuré en mode rapide 25 kHz.

9.3 Télémétrie périodique

Lorsqu'elle est activée (par défaut), une trame de télémétrie est diffusée sur le canal Bluetooth à intervalle réglable (une seconde par défaut), contenant la température relevée, le pourcentage de charge de chaque batterie et le courant instantané. L'émission de cette télémétrie est explicitement différée jusqu'à l'appel de `markSetupComplete()` en toute fin de `setup()`, afin de ne jamais l'entrelacer avec les messages d'initialisation affichés au démarrage.

10. Module Système

Le module `systeme.cpp` centralise les fonctions transverses : pilotage de la LED RGB de statut, initialisation et lecture protégée du capteur de température (garde `isTempSensorPresent()` évitant tout blocage I2C si le capteur est physiquement absent), et redémarrage logiciel contrôlé de la carte. Le redémarrage (`rebootTank()`) est réalisé

par armement du chien de garde matériel sur une durée très courte (15 millisecondes) suivi d'une boucle infinie volontaire, provoquant un reset matériel propre plutôt qu'un simple retour au début de `setup()` .

Un dispositif de protection anti chien de garde est implanté dans `main.cpp` , section `.init3` , exécuté avant même l'initialisation du framework Arduino : il efface le drapeau de reset et désactive le chien de garde dès la mise sous tension, condition nécessaire à la fiabilité du redémarrage logiciel évoqué ci-dessus sur architecture ATmega2560.

11. Référence des commandes Série et Bluetooth

Toutes les commandes suivantes sont acceptées indifféremment sur le port série USB (115200 bauds) et sur le canal Bluetooth HM-10 (9600 bauds). Les commandes marquées du badge ADMIN nécessitent un déverrouillage préalable par badge RFID autorisé ; en son absence, le firmware répond `E0AUTH` .

11.1 Informations système

Commande	Effet
<code>ping</code>	Test de disponibilité, réponse <code>pong</code>
<code>whois</code>	Affiche l'identification du projet
<code>ver arduino</code>	Affiche le modèle de carte
<code>ver firmware</code>	Affiche la version du firmware et la date de dernière compilation
<code>ver drivers</code>	Affiche la version de chaque module logiciel
<code>temp</code>	Affiche la dernière température relevée
<code>reset</code>	Réinitialise la limite de vitesse et la vitesse de ventilateur manuelle aux valeurs par défaut
<code>btbrk</code>	Déclenche une rupture volontaire de la connexion Bluetooth

11.2 Diagnostic et redémarrage ADMIN

Commande	Effet
<code>reboot arduino / reboot tank</code>	Redémarre la carte via le chien de garde matériel
<code>reboot esc</code>	Réinitialise les ESC de la tourelle
<code>reboot bt</code>	Envoie une commande AT de réinitialisation au module HM-10
<code>motor L <valeur> / motor R <valeur></code>	Active le mode diagnostic sur le moteur gauche ou droit, valeur comprise entre -100 et 100
<code>motor off</code>	Désactive le mode diagnostic moteur
<code>tare ampSensor</code>	Recalibre le zéro du capteur de courant

11.3 Thermique et ventilation

Commande	Accès	Effet
<code>fanctrl show</code>	Libre	Affiche le mode de régulation courant (OFF, MANUAL, AUTO)
<code>fanctrl auto / manual / off</code>	Libre	Force le mode de régulation du ventilateur
<code>fanspd show</code>	Libre	Affiche la vitesse de ventilateur manuelle courante
<code>fanspd set <0-100></code>	Libre	Définit la vitesse manuelle, uniquement effective en mode MANUAL

<code>fanrange show</code>	Libre	Affiche la plage thermique de régulation automatique
<code>fanrange season winter / summer</code>	ADMIN	Applique un préréglage de plage thermique saisonnier
<code>fanrange <tmin> <tmax></code>	ADMIN	Définit une plage thermique personnalisée

11.4 Vitesse et statistiques

Commande	Effet
<code>mspd show</code>	Affiche la limite de vitesse générale courante (pourcentage)
<code>mspd <0-100></code>	Définit la limite de vitesse générale appliquée à la traction
<code>mspd reset</code>	Réinitialise la limite de vitesse à 100 pourcent
<code>stats status on / off</code>	Active ou désactive la télémétrie périodique Bluetooth
<code>stats delay <secondes></code>	Règle l'intervalle d'émission de la télémétrie
<code>stats ram</code>	Affiche la marge de pile restante (high water mark) de chaque tâche FreeRTOS

11.5 Gestion des badges RFID ADMIN

Commande	Effet
<code>admin UID_AUTH show</code>	Liste les UID actuellement autorisés en mémoire programme
<code>admin UID_AUTH add</code>	Réservée pour une future implémentation d'ajout dynamique
<code>admin UID_AUTH remove</code>	Réservée pour une future implémentation de retrait dynamique

11.6 Codes de retour

Le firmware distingue trois familles de préfixes dans ses réponses : `I0xx` pour une information ou une confirmation de succès, `W0xx` pour un avertissement non bloquant, `E0xx` pour une erreur (syntaxe invalide, argument hors plage, accès refusé). Ce préfixage permet un traitement automatisé des réponses côté superviseur externe sans analyse syntaxique complexe.

12. Compilation et déploiement

Le projet est structuré pour l'environnement PlatformIO et cible l'environnement de compilation Arduino Mega 2560 (`board = megaatmega2560`). Les dépendances principales sont les suivantes :

- `feilipu/FreeRTOS` (portage AVR de FreeRTOS)
- `enableInterrupt` (capture d'impulsions radio en interruption)
- `MFRC522` (pilote du lecteur RFID)
- `Adafruit_MCP9808` (pilote du capteur de température)
- `Servo` (pilotage des servomoteurs et ESC)

Le fichier `FreeRTOSConfig.h` propre au projet, placé dans le dossier `include/`, est prioritaire sur la configuration par défaut fournie par la bibliothèque et définit notamment le nombre de niveaux de priorité disponibles (`configMAX_PRIORITIES`) et la taille du tas FreeRTOS. Cette dernière peut être ajustée par variable de compilation, par exemple :

```
build_flags =  
-DconfigTOTAL_HEAP_SIZE=3200
```

La syntaxe de cette variable ne tolère aucun espace de part et d'autre du signe égal, sous peine de mauvaise interprétation par la chaîne de liaison. Le débit série de la liaison USB est fixé à 115200 bauds ; celui du module Bluetooth HM-10 est fixé à 9600 bauds, configuration matérielle du module elle même.

Le cadencement de l'ordonnanceur AVR FreeRTOS repose sur une base de temps d'environ 61 Hz : toute temporisation de tâche doit rester cohérente avec cette granularité pour garantir un blocage réel et non une valeur tronquée à zéro tick.

13. Annexe : brochage complet

Fonction	Broche(s)
Relais direction moteur gauche (avant/arrière)	22 / 23
Relais direction moteur droit (avant/arrière)	24 / 25
Relais coupure alimentation gauche / droite	26 / 27
Sélection SPI MCP41100 gauche / droit (CS)	42 / 43
Radiocommande, axe X / Y droit	A8 / A9
Radiocommande, axe X / Y gauche (tourelle)	A10 / A11
Interrupteurs radio SWA / SWB / SWC / SWD	44 / 10 / 11 / 12
Potentiomètres radio VRA / VRB	13 / 45
Interrupteur d'armement tourelle	31
Servomoteurs élévation tourelle (x2)	2 / 3
ESC armement (x2)	4 / 5
Servo. rotation tourelle	6
PWM ventilateur	9
LED statut R / G / B	49 / 47 / 46
LED contrôle tourelle	48
Coupure matérielle MOSFET traction	32
Broche BRK module Bluetooth HM-10	37
Lecteur RFID RC522, sélection / reset (SS / RST)	41 / 40
Bus SPI matériel (SCK / MOSI / MISO)	52 / 51 / 50
Batterie châssis (analogique)	A1
Batterie tourelle (analogique)	A2
Capteur de courant (analogique)	A0
Capteur de température MCP9808 (I2C)	SDA / SCL